

Localización y Mapeo Simultáneos con Landmarks Visuales, Navegación Autónoma y Exploración Informativa en un TurtleBot4:

Memoria Técnica del Trabajo Práctico Final

Tomás Benavidez Esposito
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
tbenavidez@udesa.edu.ar

Naomi Couriel
Ingeniería en Inteligencia Artificial
Universidad de San Andrés
Buenos Aires, Argentina
ncouriel@udesa.edu.ar

Resumen—Presentamos la memoria técnica de un sistema robótico autónomo de localización y mapeo simultáneos (SLAM), navegación y exploración activa, desarrollado sobre la plataforma TurtleBot4 y ROS 2 Humble. El trabajo se estructura en tres etapas acopladas. En la Parte A implementamos un *Graph SLAM* sobre datos reales pregrabados (RosBag), fusionando el modelo de odometría por deltas con avistajes de marcadores *ArUco* detectados por la cámara OAK-D; una primera pasada optimiza el grafo de poses con cierre de lazo y una segunda pasada proyecta el LIDAR sobre la trayectoria corregida para obtener una grilla de ocupación métrica. En la Parte B desarrollamos la navegación autónoma (Sistema 3: grilla + landmarks de cámara) sobre Gazebo, integrando un filtro de partículas (MCL) que fusiona un modelo de haz láser con observaciones de un sensor virtual de landmarks con oclusión, un planificador global A^* con inflado y márgenes de seguridad, un seguidor *pure-pursuit* y una máquina de estados que gobierna el ciclo completo, incluyendo detección y evasión de obstáculos no mapeados. En la Parte C extendemos el sistema con una capa de misión de exploración informativa guiada por una función de utilidad y un módulo de percepción que detecta conos rojos por segmentación en HSV con validación geométrica y temporal, re-proyectando la detección al planificador para respetar los muros reales. Reportamos los resultados cuantitativos obtenidos en simulación y sobre los RosBags del TurtleBot4 — error de seguimiento del MCL de 0,02–0,18 m y evasión sin colisión — y documentamos las decisiones de diseño que hicieron converger cada subsistema. La validación sobre el robot físico (Parte C) queda pendiente de la ventana de laboratorio y se detalla como trabajo a completar.

Index Terms—SLAM, Graph SLAM, ArUco, Monte Carlo Localization, filtro de partículas, navegación autónoma, A^* , máquina de estados, exploración informativa, ROS 2, TurtleBot4.

I. INTRODUCCIÓN

La localización y el mapeo simultáneos (SLAM) constituyen uno de los problemas fundamentales de la robótica móvil: un robot que se desplaza por un entorno desconocido debe, al mismo tiempo, construir un mapa del entorno y estimar su propia posición dentro de él, empleando únicamente sensores inexactos y un modelo de movimiento afectado por ruido. Este trabajo práctico final integra los principales

enfoques probabilísticos estudiados a lo largo de la materia *Principios de la Robótica Autónoma* en un sistema completo que abarca desde la estimación de estado hasta la actuación autónoma.

El sistema se desarrolla en tres etapas encadenadas, en las que cada una consume el producto de la anterior:

- **Parte A — SLAM.** A partir de datos reales pregrabados del TurtleBot4 (RosBag con LIDAR, cámara y odometría), el robot recorre un laberinto y construye una representación del entorno mientras estima su trayectoria. Elegimos la *Opción 3* de la consigna: extracción de características mediante marcadores *ArUco* y *Graph SLAM* obligatorio con cierre de lazo. El entregable es una grilla de ocupación métrica más el mapa geométrico de landmarks visuales indexados por su ID.
- **Parte B — Navegación autónoma.** Sobre el mapa consolidado, el robot se localiza de forma continua con un filtro probabilístico y navega de manera segura entre puntos arbitrarios seleccionados por el usuario, planificando rutas libres de colisión, siguiéndolas suavemente, alcanzando el ángulo final, re-planificando ante nuevos objetivos y evadiendo obstáculos no mapeados. Todo el comportamiento se gobierna mediante una máquina de estados.
- **Parte C — Despliegue real y exploración.** El sistema se lleva a la plataforma física y se le añade una misión de exploración activa: recorrer autónomamente el laberinto buscando conos rojos, discriminándolos de distractores de otros colores y aproximándose a ellos sin intentar atravesar paredes.

Nuestra contribución principal no es un algoritmo nuevo, sino la *integración rigurosa* de estas técnicas en un sistema funcional y modular, y la documentación honesta de las decisiones de diseño — y de los errores corregidos — que fueron necesarios para que cada subsistema convergiera sobre datos reales y ruidosos. Como pide la consigna, incluimos los

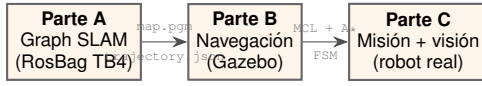


Figura 1. Encadenamiento de las tres partes. La Parte A produce la grilla de ocupación y el mapa de landmarks ArUco; la Parte B aporta la pila de localización, planificación y control; la Parte C reutiliza ambas y añade la capa de misión y percepción sobre el robot físico.

diagramas de bloques de las máquinas de estados empleadas en las Partes B y C.

La Fig. 1 resume el flujo de datos entre las tres partes. Advertencia metodológica: al momento de redacción no se realizó todavía la prueba sobre el robot físico; los resultados de la Parte C sobre hardware real se marcan explícitamente como *a completar* y se dejan preparados los procedimientos de validación (RosBags y gates de seguridad).

II. MARCO TEÓRICO Y PLATAFORMA

II-A. Enfoque probabilístico

Adoptamos el marco probabilístico clásico de la robótica [1]. El estado del robot en el plano es la pose $\mathbf{x}_t = (x, y, \theta)^\top$. La creencia sobre el estado se propaga con el modelo de movimiento por odometría $p(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{u}_t)$ y se corrige con el modelo de medición $p(\mathbf{z}_t | \mathbf{x}_t, \mathbf{m})$ dado el mapa $\mathbf{m}$. Según la etapa, la inferencia sobre esta creencia se resuelve por optimización de un grafo de factores (Parte A) o mediante un filtro de partículas (Parte B/C).

II-B. Modelo de movimiento por odometría

En las tres partes, la predicción se basa en el modelo de *deltas* de odometría de Thrun [1]: entre dos lecturas consecutivas de odometría se descompone el desplazamiento relativo en una rotación inicial δ_{rot1} , una traslación δ_{trans} y una rotación final δ_{rot2} ,

$$\delta_{\text{trans}} = \sqrt{(\bar{x}' - \bar{x})^2 + (\bar{y}' - \bar{y})^2}, \quad (1)$$

$$\delta_{\text{rot1}} = \text{atan2}(\bar{y}' - \bar{y}, \bar{x}' - \bar{x}) - \bar{\theta}, \quad (2)$$

$$\delta_{\text{rot2}} = \bar{\theta}' - \bar{\theta} - \delta_{\text{rot1}}, \quad (3)$$

computados siempre como diferencia respecto de la odometría del instante anterior. El ruido se modela como gaussiano con parámetros $\alpha_1 \dots \alpha_4$ sobre cada componente.

II-C. Plataforma y entorno de trabajo

El robot es un TurtleBot4 (base iRobot Create 3, LIDAR 2D RPLIDAR y cámara estéreo OAK-D). Para la Parte A trabajamos con RosBags reales grabados con el TurtleBot4 (namespace `tb4_1/tb4_0`); para la Parte B usamos el simulador Gazebo con el mundo `custom_casa`; y la Parte C está preparada para el robot físico y validada previamente contra RosBags. Todo el software se implementó en ROS 2 Humble, organizado en paquetes `ament_python` independientes por parte (`tp_a_slam_aruco`, `tp_b_navigation`, `tp_c_mission`) más paquetes compartidos de interfaces y utilidades.

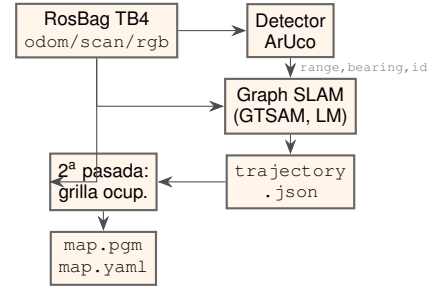


Figura 2. Pipeline de dos pasadas de la Parte A. La primera pasada consolida el grafo de poses (odometría + avistajes ArUco) y exporta la trayectoria optimizada; la segunda re-proyecta el LIDAR sobre la trayectoria corregida para generar la grilla de ocupación.

III. PARTE A: SLAM VISUAL CON GRAPH SLAM Y MARCADORES ARUCO

III-A. Elección del enfoque

La consigna ofrece tres caminos de implementación (Tabla I). Elegimos la **Opción 3: features con cámara**, que trabaja sobre un RosBag con datos reales del TurtleBot4 y exige *Graph SLAM* con cierre de lazo. Esta opción es la más realista y la más exigente: no dispone de *ground truth*, sufre *motion blur* y baja densidad de marcadores en algunas zonas, y obliga a corregir la deriva odométrica mediante la re-observación de landmarks. A cambio, produce un mapa geométrico de marcadores indexado por ID que las Partes B y C reutilizan directamente para localizar el robot físico.

Cuadro I
OPCIONES DE IMPLEMENTACIÓN DE LA PARTE A

Opción	Entorno	Sensores	SLAM
1	Gazebo	LIDAR	Grid FastSLAM
2	Gazebo	LIDAR	EKF/Graph (features)
3	RosBag TB4	Cámara+LIDAR	Graph SLAM

El procesamiento es de **dos pasadas** sobre el mismo bag (Fig. 2): la primera optimiza el grafo de poses y landmarks para obtener la trayectoria corregida; la segunda proyecta el LIDAR sobre esa trayectoria para construir una grilla de ocupación consistente, evitando así la distorsión que produciría mapear con la odometría cruda.

III-B. Detección de marcadores y modelo de medición

El nodo detector emplea OpenCV [4] con el diccionario `DICT_4X4_50` y un lado de marcador de 0,0889 m. La pose 3D de cada tag se estima con `solvePnP` usando el solucionador `IPPE_SQUARE`, óptimo para marcadores planos cuadrados, a partir de la matriz intrínseca K y los coeficientes de distorsión D provistos por la cátedra (calibración de la cámara del TB4 #0). Cada detección devuelve la traslación $\mathbf{t} = (t_x, t_y, t_z)$ en el marco óptico de la cámara y el error de reproyección RMS.

Filtrado de detecciones (robustez ante datos reales): descartamos las detecciones poco confiables antes de que entren al grafo, mitigando *motion blur* y falsos positivos:

- área proyectada del marcador < 250 px (tags lejanos o borrosos);
- error de reproyección $> 4,0$ px (ajuste PnP inconsistente por desenfoque);
- profundidad t_z fuera de $[0,05, 3,0]$ m.

De detección a observación range-bearing: la traslación en el marco óptico se transforma al plano del robot. Se prefiere la TF real del bag (`base_link ← camera`); si no está disponible dentro de la tolerancia, se recurre a los extrínsecos fijos del montaje del TB4 ($t_x = -0,0596$ m, $\text{yaw} = 0$), mapeando el marco óptico (z adelante, x derecha) al plano como $(t_z, -t_x)$. Con la posición (x_b, y_b) del marcador en el marco base se calcula la medición

$$r = \sqrt{x_b^2 + y_b^2}, \quad \phi = \text{atan2}(y_b, x_b). \quad (4)$$

Modelo de ruido dependiente de la distancia: caracterizamos empíricamente (con la grabación `aruco_estimation` de distancias controladas) las desviaciones estándar de la medición como funciones del rango,

$$\sigma_\phi(r) = 0,10 + 0,05 r \quad [\text{rad}], \quad (5)$$

$$\sigma_r(r) = 0,0239 + 0,0315 r^2 \quad [\text{m}], \quad (6)$$

de modo que las observaciones lejanas (más ruidosas por resolución angular y desenfoque) pesan menos en la optimización. La matriz de información del factor es $\text{diag}(\sigma_\phi^{-2}, \sigma_r^{-2})$.

III-C. Formulación del Graph SLAM

Implementamos el *backend* sobre **GTSAM**, construyendo un grafo de factores no lineal [2]. Las variables son las poses del robot $X(i) \in SE(2)$ (`Pose2`, una por *keyframe*) y los landmarks $L(\text{id}) \in \mathbb{R}^2$ (`Point2`), donde la clave de cada landmark es su propio ID ArUco. Se crea un nuevo *keyframe* cuando el robot se desplaza $\geq 0,15$ m o gira $\geq 0,60$ rad.

El grafo contiene tres tipos de factores:

1. **Prior** sobre $X(0)$ anclando el origen, con $\sigma = [0,01, 0,01, 0,005]$.
2. **Odometría** (`BetweenFactorPose2`) entre poses consecutivas. La pose relativa se calcula con la operación del grupo de Lie $T_{i-1}^{-1}T_i$ (en vez de la descomposición manual de deltas), lo que es numéricamente más robusto ante giros grandes; su ruido $\sigma = [0,3, 0,3, 0,1]$ se elige deliberadamente *generoso* para que las restricciones de landmark tengan autoridad para corregir la deriva.
3. **Avistaje** (`BearingRangeFactor2D`) entre $X(i)$ y $L(\text{id})$, con el ruido dependiente de la distancia.

La optimización minimiza la suma de residuos de Mahalanobis al cuadrado

$$\mathbf{X}^* = \arg \min_{\mathbf{X}} \sum_k \mathbf{e}_k(\mathbf{X})^\top \Omega_k \mathbf{e}_k(\mathbf{X}), \quad (7)$$

resuelta con **Levenberg–Marquardt** (GTSAM linealiza cada factor con jacobianos analíticos). Tras cada optimización se

reconstruye la estimación inicial desde el resultado (*warm-start*). Para robustez ante outliers, los factores de avistaje se envuelven en un **kernel robusto de Cauchy** ($k = 1,0$), que satura el aporte de mediciones aberrantes en lugar de dejarlas crecer cuadráticamente.

Inicialización de landmarks: un marcador nuevo se agrega al grafo sólo tras observarse ≥ 3 veces, con posición inicial $\mathbf{x}_i + r(\cos(\theta_i + \phi), \sin(\theta_i + \phi))$.

III-D. Cierre de lazo por re-observación

No usamos un *loop closure* geométrico por *scan-matching*; el cierre de lazo es **implícito por re-observación de landmarks**: cuando un marcador de ID ya conocido se vuelve a ver desde una pose distante en el tiempo, se agrega un nuevo factor de avistaje que ata ambas poses al mismo $L(\text{id})$, corrigiendo la deriva acumulada al regresar a zonas visitadas. Para evitar falsos cierres, cada re-observación debe superar tres computas:

- **Paralaje mínimo** de 0,20 m desde la última observación del landmark (evita restricciones redundantes sin información);
- **Salto espacial**: se rechaza si la posición del landmark predicha desde la observación se aparta $> 0,75$ m de su estimación actual;
- **Innovación de Mahalanobis** $\chi_2^2 < 5,99$ (umbral al 95 %), sobre $(\Delta\phi, \Delta r)$.

Además se limita a 50 el número de factores por landmark para acotar el tamaño del grafo.

III-E. Sincronización temporal

Las detecciones ArUco y la odometría llegan con estampas de tiempo distintas. Interpolamos la pose de odometría al instante exacto de cada detección mediante *bracketing*: se buscan las dos muestras de odometría que rodean la estampa y se interpola en $SE(2)$ (traslación lineal, ángulo por el camino corto). El diseño **nunca extrapola**: si no hay muestra a ambos lados, la detección espera; si envejece más de 1,0 s sin *bracket*, se descarta. Este mismo mecanismo se aplica en la segunda pasada para asignar a cada barrido LIDAR su pose corregida.

III-F. Segunda pasada: grilla de ocupación

Con la trayectoria optimizada, generamos la grilla mediante un *inverse sensor model* con **log-odds**. La clave para un mapa nítido es la pose asignada a cada barrido: usamos un modo *denso* que compone la corrección de SLAM interpolada entre *keyframes* con la odometría densa de cada instante, $T_{\text{map}}(t) = T_{\text{corr}}(t) \circ T_{\text{odom}}(t)$, en lugar de interpolar linealmente poses dispersas.

Cada rayo del LIDAR se traza con **Bresenham**: las celdas atravesadas reciben $L_{\text{free}} = -0,40$ y la celda de impacto $L_{\text{occ}} = +0,85$ (asimétrico, conservador al limpiar), saturando en $[-5, +5]$. La resolución es 0,05 m/celda; se limpia espacio libre hasta 2,5 m y se marca ocupado sólo hasta 2,0 m. Los extrínsecos del LIDAR se leen de la TF; el *fallback* fijo ($t_x = -0,04$, $\text{yaw} = +\pi/2$) refleja que el RPLIDAR está montado rotado 90° respecto de `base_link` (ignorarlos)

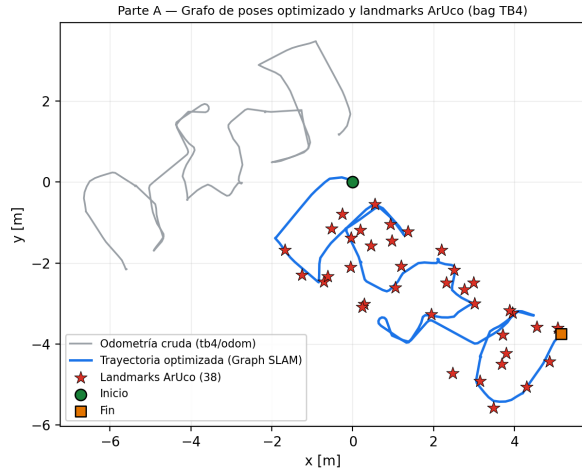


Figura 3. Parte A: grafo de poses optimizado (azul) frente a la odometría cruda del TB4 (gris) y los 38 landmarks ArUco estimados (estrellas rojas). La deriva odométrica de varios metros se corrige por las re-observaciones de marcadores.

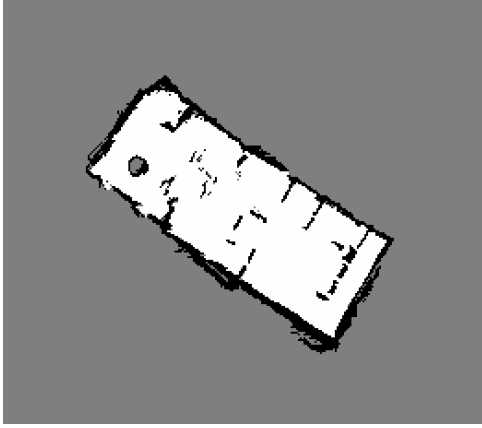


Figura 4. Grilla de ocupación del laberinto real (bag TB4) generada en la segunda pasada: 262×231 celdas a 0,05 m. Negro: ocupado; blanco: libre; gris: desconocido.

emborrona las paredes). Un **gate angular** descarta los barridos tomados mientras $|\omega| > 0,6$ rad/s, cuando la pose angular es menos confiable. La grilla se exporta a formato nav2 `map_server (.pgm+.yaml, umbrales 0,60/0,40)`.

III-G. Resultados

Sobre el bag del laberinto del TB4, la primera pasada consolidó **277 poses optimizadas** y **38 landmarks ArUco** únicos (IDs 0–47 con huecos). La Fig. 3 contrasta la odometría cruda con la trayectoria optimizada: la odometría acumula una deriva de varios metros, mientras que las restricciones de landmark cierran el recorrido de forma coherente. La segunda pasada produjo una grilla de 262×231 celdas a 0,05 m (Fig. 4), con las paredes principales continuas y las aberturas transitables preservadas.

Para verificar la estabilidad del mapeo evaluamos el efecto del *bracketing* de TF y del gate angular sobre un bag largo (1392.8 s, 10 797 barridos). La Tabla II resume la comparación entre desactivar y activar el gate angular: ambas variantes integran prácticamente todos los barridos usando la TF real (un único *fallback* por corrida), producen la misma cantidad de celdas ocupadas y difieren en una sola celda, confirmando que el *bracketing* elimina el *clamp* al último odom sin necesitar cambios de log-odds. Se conserva el gate desactivado por defecto.

Cuadro II
ESTABILIDAD DEL MAPEO (BAG LARGO, 10 797 BARRIDOS)

Métrica	Sin gate	Gate 0.35
Barridos integrados	10 770	10 783
Descartados por giro	0	1
TF real / fallback	10 769/1	10 782/1
Máximo gap odométrico	946.9 ms	590.1 ms
Celdas ocupadas	2 058	2 058
Espesor de pared (mediana/p95)	0.10/0.28 m	0.10/0.28 m
Celdas distintas entre mapas		1

La Tabla III reúne los parámetros de diseño más relevantes de la Parte A.

Cuadro III
PARÁMETROS CLAVE DE LA PARTE A

Módulo	Parámetro	Valor
Keyframe	distancia / ángulo	0.15 m / 0.60 rad
Odometría	$\sigma(x, y, \theta)$	0.3, 0.3, 0.1
Avistaje	$\sigma_\phi(r), \sigma_r(r)$	$0,10+0,05r / 0,024+0,032r^2$
Optimizador	método / kernel	LM / Cauchy $k=1$
Cierre lazo	paralaje / salto / χ^2	0.20 m / 0.75 m / 5.99
ArUco	área mín / reproj. máx	250 px / 4.0 px
Grilla	resolución	0.05 m/celda
Grilla	L_{occ}/L_{free}	+0,85 / -0,40

IV. PARTE B: NAVEGACIÓN AUTÓNOMA (SISTEMA 3)

La Parte B implementa la navegación autónoma sobre el mapa consolidado, siguiendo el *Sistema 3* de la consigna (grilla de ocupación + landmarks de cámara). Como Gazebo no provee marcadores visuales nativos, diseñamos un **sensor virtual de landmarks** que emula el detector ArUco. Todo el código nuevo reside en el paquete `tp_b_navigation`, sin modificar la Parte A.

IV-A. Arquitectura de nodos y cadena de TF

El sistema se descompone en siete nodos con responsabilidades claras (Fig. 5). La decisión de diseño central es que **un único nodo, la máquina de estados, es el dueño exclusivo de /cmd_vel**: fusiona la FSM con el controlador de seguimiento, el ángulo final y la evasión, porque coordinar el control entre procesos es mucho más frágil que tener un único productor de velocidad. El planificador y el monitor de obstáculos quedan como nodos puros, separados para claridad.

La cadena de transformadas es `map → odom → base_footprint → base_scan`. Gazebo publica

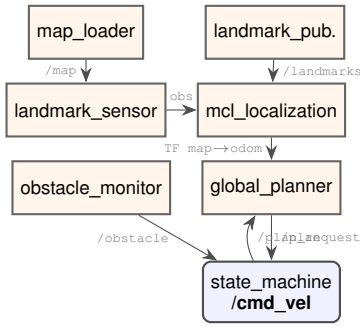


Figura 5. Mapa de nodos de la Parte B. El MCL cierra la cadena de TF publicando `map→odom`; la máquina de estados es el único productor de `/cmd_vel`.

`odom→base_footprint`; el **MCL cierra la cadena** publicando la corrección `map→odom` a 30 Hz. Todos los consumidores leen la pose del robot con `lookup_transform(map, base_footprint)`.

IV-B. Localización: filtro de partículas (MCL)

El nodo `mcl_localization` implementa *Monte Carlo Localization* [5] con mapa y landmarks conocidos y fijos (no hace SLAM). Usa $N = 350$ partículas, inicializadas por la herramienta “2D Pose Estimate” de RViz (`/initialpose`).

Predicción: por cada mensaje de `/calc_odom` se aplica el modelo de movimiento por odometría (deltas δ_{rot1} , δ_{trans} , δ_{rot2} con ruido $\alpha_{1..4}$). La odometría de referencia de Gazebo se mantiene separada sólo para construir la TF `map→odom`.

Corrección híbrida: las partículas se pesan combinando dos modelos de medición:

- **Láser–mapa:** se construye un *likelihood field* desde las celdas ocupadas del mapa y se pondera según cuán cerca caen de las paredes los *endpoints* muestreados del LIDAR (hasta 60 haces, $\sigma_{hit} = 0,18$ m). Partículas fuera del mapa, en desconocido o dentro de ocupado reciben una penalización fuerte.
- **Landmarks:** acepta tanto el contrato por índice (`/observed_landmarks`) como el identificado por ID ArUco (`/observed_landmark_ids`); en el bag/real la asociación es directa contra los IDs del `trajectory.json` de la Parte A.

Las correcciones se disparan sólo tras movimiento suficiente (`update_min_d/update_min_a`) para evitar la degeneración estática de la nube.

Remuestreo y diversidad: se usa remuestreo de baja varianza cuando $n_{eff} < N/2$, seguido de *roughening* (jitter 0,03 m en posición, 0,02 rad en ángulo) para evitar el empobrecimiento de partículas. Finalmente, la TF `map→odom` se publica con la estampa adelantada 0,1 s (`transform_tolerance`, truco de AMCL) para que siga siendo válida ante lecturas con estampa más nueva, evitando el parpadeo por “extrapolation into the future”.

IV-C. Sensor virtual de landmarks con oclusión

El nodo `landmark_sensor` emula el detector ArUco ausente en Gazebo. Por cada landmark conocido: (1) lo proyecta al marco del robot y calcula *range+bearing*; (2) descarta los que caen fuera del FOV de 1,05 rad o del rango; (3) calcula la **oclusión por línea de visión** comparando el *range* esperado contra el rayo central del LIDAR (si un obstáculo aparece $> 0,08$ m antes, el landmark está ocluido y no se publica); (4) agrega ruido gaussiano ($\sigma_r = \sigma_\phi = 0,05$).

Dos decisiones fueron críticas para la convergencia:

- **Desacople de la estimación (`truth_frame`):** una cámara real ve los landmarks según la pose *verdadera* del robot, no según la estimación del filtro. Proyectar con la TF del MCL realimentaría el filtro consigo mismo y lo haría diverger. Por eso el sensor proyecta con `odom→base` (en Gazebo la odometría es cuasi *ground-truth*).
- **Estampa de la TF, no del scan:** la salida se estampa con el tiempo de la TF usada, no con el del `/scan` (que Gazebo adelanta 20–60 ms), garantizando una cadena `map→odom→base` válida en ese instante.

Densidad de landmarks: la consigna pide una densidad “coherente con el ArUco real”. Escalamos de 14 a 36 y finalmente a **60 landmarks**, distribuidos por *farthest-point sampling* sobre superficies de pared (un landmark en espacio libre nunca devolvería eco del LIDAR y jamás sería “visible”). Con pocos landmarks el posterior se volvía multimodal por las simetrías de la casa (§IV-G).

IV-D. Planificación global: A*

El `global_planner` ejecuta A* 8-conexo [6] sobre la grilla, con: **inflado** de obstáculos por el radio del robot (0,18 m, distancia de Chebyshev por BFS multi-fuente); **penalización de cercanía** (*clearance*) que encarece pasar pegado a las paredes, de modo que el robot prefiere el centro de los pasillos; tratamiento de las celdas desconocidas como obstáculo (`allow_unknown=False`, por seguridad); y un **atajo por línea de visión** (Bresenham) que simplifica la ruta a pocos *waypoints*, forzando el último exactamente al `goal_pose`. Es un nodo puro: planifica al recibir `/plan_request` y publica `/plan`.

IV-E. Seguimiento, ángulo final y obstáculos

El seguimiento usa **pure-pursuit** [7] con un punto “zanahoria” a `lookahead = 0,30` m: el rumbo se corrige con ganancia $k_w = 1,6$ y la velocidad lineal se reduce con el error angular, $v = v_{\max} \max(0, 1 - |e|/\text{slow_angle})$, de modo que el robot gira primero y avanza suave ($v_{\max} = 0,18$ m/s, $w_{\max} = 1,2$ rad/s). Alcanzada la posición (`goal_xy_tol = 0,12` m), un estado dedicado gira en el lugar hasta el **ángulo final** (`goal_yaw_tol = 0,08` rad).

Para **obstáculos no mapeados**, el `obstacle_monitor` distingue paredes conocidas de obstáculos nuevos: por cada eco del LIDAR en un cono frontal (0,6 rad) y cercano ($< 0,45$ m), proyecta el punto a `map` y consulta la

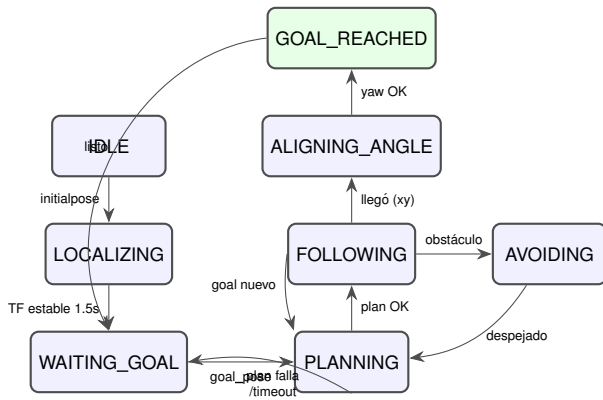


Figura 6. Diagrama de bloques de la máquina de estados de la Parte B. Cada arco indica el evento o condición que dispara la transición. Un /initialpose (no dibujado desde todos los orígenes) reinicia a LOCALIZING, y un goal_pose nuevo re-planifica desde cualquier estado activo.

celda; si el mapa la marca libre pero el LIDAR ve algo, es un obstáculo nuevo. Con ≥ 3 puntos nuevos publica /obstacle_detected, y conserva los puntos en /dynamic_obstacles (con inflación compacta de 0,08 m) para que el siguiente A* los rodee sin clausurar todo el pasillo.

IV-F. Máquina de estados

La Fig. 6 muestra el diagrama de bloques de la máquina de estados de la Parte B, que gobierna el ciclo completo con ocho estados. El flujo nominal es IDLE → LOCALIZING → WAITING_GOAL → PLANNING → FOLLOWING → ALIGNING_ANGLE → GOAL_REACHED. Las ramas cubren la evasión (AVOIDING), el re-planeo ante un nuevo objetivo (re-planifica) y el fallo/timeout de planificación. Un /initialpose reinicia siempre a LOCALIZING.

IV-G. Resultados y bitácora de tuning

Ciclo completo en custom_casa. Con localización perfecta (TF identidad, para aislar la navegación), el robot fue a un goal a 180° y llegó con error ~ 10 cm/3°, rodeando correctamente la sala central. Con el MCL real, cruzó toda la casa (~ 5 m) y llegó con error ~ 6 cm, con el MCL estimando a ~ 1.5 cm de la pose real en el goal (Fig. 7).

Convergencia del MCL. El MCL inicialmente divergía. Tres correcciones, cada una verificada con datos, lo estabilizaron (Tabla IV): (1) desacoplar el sensor de la estimación; (2) elevar la densidad de 14 a 36–60 landmarks para eliminar la multimodalidad; (3) roughening + 350 partículas para recuperarse de discrepancias transitorias en lugar de diverger.

Cuadro IV
TUNING DEL MCL: ANTES VS. DESPUÉS

Config.	Err. medio	Err. máx	Comportamiento
Antes (14 lm, acoplado)	0.65 m	2.9 m	diverge
Después (36–60 lm, rough.)	0.02–0.18 m	recupera	trackea pegado

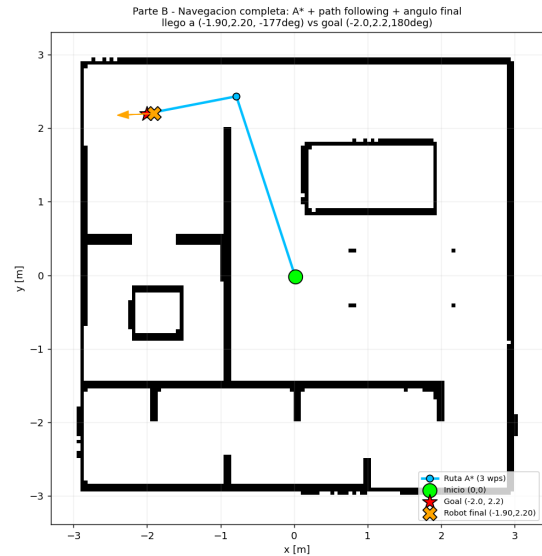


Figura 7. Navegación en custom_casa: ruta A* (verde) que rodea la sala central, trayectoria seguida y goal alcanzado con ángulo final.

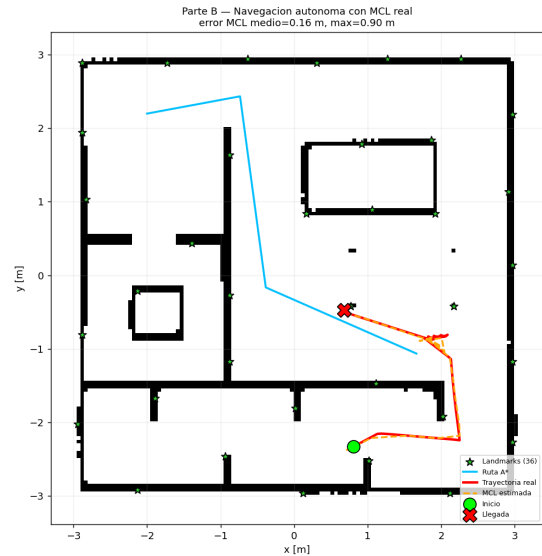


Figura 8. Evasión en custom_casa_obs: el robot detecta la valija no mapeada, la esquiva (trayectoria roja) sin colisión y el MCL (naranja) se mantiene pegado a la pose real.

Obstáculos no mapeados en custom_casa_obs. Con un goal cuya ruta cruza una valija no mapeada, el monitor la detectó correctamente, la FSM entró en AVOIDING (8 maniobras), el robot **esquivó sin chocar** y el MCL se mantuvo preciso durante las rotaciones (error medio 0,16 m, máx 0,90 m sobre 799 muestras). Con goals que no cruzan obstáculos no hubo falsos positivos (Fig. 8).

Limitación conocida. Cuando un obstáculo se sienta *justo* sobre la única ruta buena (p.ej. un cluster tapando la entrada al cuarto del goal), la navegación se vuelve lenta y sinuosa: el planificador razona sobre el mapa estático y re-planifica

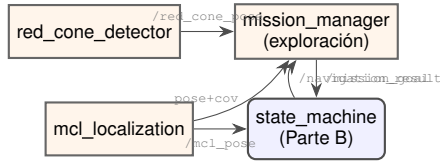


Figura 9. Arquitectura de la Parte C. La misión decide *a dónde* ir y la FSM de la Parte B decide *cómo* moverse; la realimentación viaja por /navigation_result.

la misma ruta bloqueada, avanzando sólo por los empujones reactivos de AVOIDING. No choca ni se rompe, pero es ineficiente. La mejora natural es un *costmap local* que sume los obstáculos detectados al mapa estático para que A* los rodee. La Tabla V resume el cumplimiento de la consigna.

Cuadro V
CUMPLIMIENTO DE LA CONSIGNA DE PARTE B

Requisito	Estado
1.1 Sistema 3 (landmarks de cámara)	✓ 60 lm + oclusión
1.2 Localización inicial (initialpose)	✓
1.4 Localización continua (filtro prob.)	✓ MCL 0.02–0.18 m
1.5 Planificación (A*/inflado/seguro)	✓
1.6 Seguimiento suave	✓ pure-pursuit
1.7 Ángulo final	✓
1.8 Re-planeo (goal nuevo)	✓
1.9 Obstáculos no mapeados	△ esquivar; ineficiente si tapa ruta
1.11 Máquina de estados	✓ FSM completa

Re-mapeo del entorno simulado. Como pide el Sistema 3, no reutilizamos el mapa del RosBag (mundo distinto), sino que repetimos el mapeo en Gazebo con el nodo `sim_mapper`, que porta el núcleo de la grilla de ocupación de la Parte A (mismas constantes de log-odds y export) pero toma la pose de cada barrido de la TF en vivo. El mapa generado por teleoperación (220 × 220 a 0,05 m) cubre el 100% de las paredes con alineación casi perfecta respecto del mapa de referencia.

V. PARTE C: DESPLIEGUE REAL, EXPLORACIÓN INFORMATIVA Y PERCEPCIÓN

La Parte C lleva el sistema al robot físico y le añade una **misión de exploración activa**: recorrer autónomamente el laberinto buscando conos rojos, discriminándolos de distractores de otros colores y aproximándose sin intentar atravesar paredes. La capa de misión (paquete `tp_c_mission`) se monta *encima* de la navegación estable de la Parte B sin reemplazarla: el `mission_manager` nunca produce `/cmd_vel`, sino que publica objetivos en `/mission_goal` y deja que la máquina de estados de la Parte B ejecute el movimiento (Fig. 9).

V-A. Exploración informativa

El `mission_manager` mantiene una creencia de cobertura (CoverageBelief) de qué celdas libres ya fueron observadas por la cámara. En cada ciclo genera poses candidatas alcanzables (barrido de la grilla con paso 0,50 m, 8 orientaciones por celda) y, para cada una, estima por *ray-casting*

qué celdas vería la cámara (FOV 1,05 rad, alcance 3,0 m; la visibilidad se precalcula una vez por mapa). Selecciona la acción que maximiza una **función de utilidad**:

$$U = 0,55 g_{\text{cob}} + 0,20 g_{\text{loc}} s_{\text{cov}} - 0,20 c_{\text{cam}} - 0,05 (\rho + \text{rep}), \quad (8)$$

donde g_{cob} es la ganancia de cobertura (celdas nuevas normalizadas), g_{loc} la fracción de landmarks de la Parte A que quedarían visibles (mejora la localización), s_{cov} un factor que pesa más la localización cuando la incertidumbre del MCL es alta, c_{cam} el costo del camino A* normalizado, ρ el riesgo por cercanía a obstáculos y rep una penalización a celdas ya visitadas. El costo del camino se refina con A* sólo para los mejores candidatos por un *ranking* barato previo. Si no hay acciones informativas o la cobertura supera el 95 %, la misión termina en NOT_FOUND; existe además un *fallback* de frontera que prioriza celdas libres no observadas adyacentes a lo ya visto.

V-B. Detección de conos rojos

El nodo `red_cone_detector` resuelve los dos desafíos que plantea la consigna: discriminar el rojo de distractores y no “ver a través” de las paredes.

Segmentación en HSV (dos bandas): el rojo envuelve el origen del matiz, por lo que la máscara combina dos bandas ($H \leq 10$ o $H \geq 170$ en escala OpenCV) con saturación ≥ 100 y valor ≥ 70 . La máscara se limpia con apertura + cierre morfológico.

Validación geométrica: sobre las componentes conexas se exige área ≥ 150 px, altura ≥ 10 px, un factor de llenado $\geq 0,20$ y **forma de cono** (base más ancha que la punta). Con *depth* real se añade validación 3D que descarta distractores planos (como una superficie roja): se estima la altura real del objeto y se rechaza si difiere $> 0,13$ m de los $\sim 0,30$ m del cono, o si su base no está apoyada en el piso ($|z| > 0,20$ m).

Confirmación temporal: un rastreador exige ≥ 3 detecciones en una ventana de 5 *frames* con centros consistentes (< 25 px de salto) antes de confirmar, filtrando destellos espurios.

Estimación de distancia (con degradación elegante): se prefiere la mediana del *depth* alineado sobre los píxeles del cono ($\sigma = 0,05$ m); si no hay *depth* válido, se usa el *fallback* monocular $d = f_y h_{\text{cono}} / h_{\text{px}}$ ($\sigma = 0,20$ m); en el robot real se usan además haces del LIDAR en la dirección del cono. La detección confirmada se publica como `/red_cone_pose` en el marco `map`, con covarianza acorde a la fuente.

Crucialmente, la detección **nunca se convierte en un comando de velocidad**: reporta la coordenada al planificador, que genera una trayectoria válida que rodea los muros reales (si el cono se ve por una abertura, el objetivo se valida sobre la grilla). Sólo se acepta una pose de aproximación libre, visible y alcanzable por A* (anillo de *standoff* 0,55 m alrededor del cono).

V-C. Adaptador ArUco→MCL para el robot real

En el robot físico no hay sensor virtual: las detecciones ArUco reales las procesa `aruco_mcl_adapter`, que transforma cada marcador a *range/bearing* con su ID y compensa

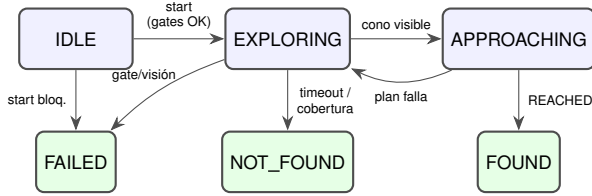


Figura 10. Diagrama de bloques de la máquina de estados de la misión de la Parte C. Los estados verdes son terminales. La cancelación (/mission/cancel) retorna a IDLE desde cualquier estado activo (no dibujada).

la latencia de la imagen. Un hallazgo de laboratorio fue que la imagen RGB podía llegar al *callback* 2.6–2.9 s después de su estampa, mientras el LIDAR llegaba fresco; el adaptador re-expresa la observación de $base(t_{obs})$ a $base(t_{ahora})$ usando el movimiento relativo en *odom* antes de publicarla. El MCL asocia entonces la observación por ID directamente contra las posiciones optimizadas del *trajectory.json* de la Parte A, sin *data association* geométrica.

V-D. Máquina de estados de la misión

La Fig. 10 muestra el diagrama de bloques de la máquina de estados de la misión (Parte C). Desde IDLE, el servicio /mission/start — que sólo procede si hay mapa, pose MCL fresca y visión lista — pasa a EXPLORING. Una detección confirmada alcanzable dispara APPROACHING; al alcanzar el cono (/navigation_result = REACHED) la misión termina en FOUND. Un plan fallido de aproximación vuelve a EXPLORING. Los estados terminales NOT_FOUND (timeout o cobertura agotada) y FAILED (gate de seguridad, ver abajo) cierran la misión y detienen la navegación.

V-E. Gates de seguridad

En el perfil real, la misión aborta a FAILED y publica velocidad cero si la pose del MCL queda vieja ($> 1,0$ s), su covarianza supera los umbrales ($0,25 \text{ m}^2$ en posición, $0,5$ en yaw) o la visión/TF de cámara deja de estar fresca ($> 3,0$ s). Estos *gates* también bloquean el arranque de la misión. Es una salvaguarda deliberada para el hardware: ante duda sobre dónde está el robot, se detiene.

V-F. Validación y estado (a completar)

Siguiendo la metodología exigida por la consigna, validamos exhaustivamente contra RosBags antes del laboratorio. Los tests unitarios de percepción, exploración y contratos pasan sin ROS; el *smoke* conjunto sobre el bag del laberinto (transporte, predicción del MCL y salud del monitor) aprueba 3 pruebas y la suite ROS 215 pruebas, sin publicar jamás sobre el /cmd_vel físico. La Fig. 11 ilustra la cobertura explorada y los *frontiers* candidatos en simulación.

Pendiente de hardware. Al momento de esta redacción no se realizó todavía la prueba sobre el robot físico. Quedan pendientes los *gates* físicos, que se ejecutarán en la ventana de laboratorio con el robot detenido y la parada de emergencia accesible: (B1) localización con movimiento deshabilitado;

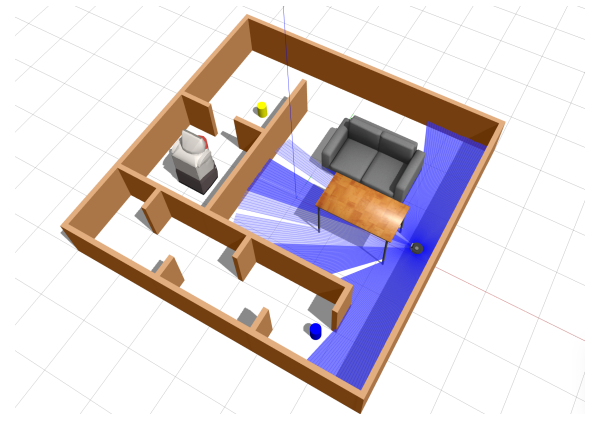


Figura 11. Exploración informativa en simulación: cobertura observada por la cámara (celdas) y candidatos *frontier* evaluados por la función de utilidad.

(B2) un *goal* corto a velocidad reducida y luego múltiples *goals*; (B3) detección de obstáculo con el robot detenido, replano sin movimiento y evasión reducida con recuperación del MCL. El análisis cuantitativo de la brecha *sim-to-real* y los resultados de la misión de conos sobre hardware se incorporarán tras esa sesión (§VI).

VI. DISCUSIÓN Y BRECHA SIM-TO-REAL

El hilo conductor de las tres partes fue la **gestión rigurosa de la incertidumbre y del tiempo**. Varias de las decisiones de diseño más importantes no surgieron de la teoría sino de patologías observadas sobre datos reales o simulados: el desacople del sensor virtual de la estimación del filtro (sin el cual el MCL se realimentaba y divergía), el *bracketing* temporal que nunca extrapola (que eliminó el emborronamiento de las paredes en el mapa), la compensación de la latencia de la imagen ArUco (2.6–2.9 s en el laboratorio) y el kernel robusto de Cauchy sobre los avistajes (que evita que un falso cierre de lazo arruine el grafo). Documentarlas es, a nuestro juicio, tan valioso como el resultado final.

Sobre la brecha *sim-to-real*. La arquitectura fue diseñada explícitamente para minimizar esta brecha: la Parte B corre sobre landmarks *virtuales* calibrados en densidad y con oclusión geométrica para plantear “un desafío matemático equivalente” al del ArUco real, y el mismo nodo MCL acepta tanto los IDs virtuales de simulación como los IDs ArUco reales del *trajectory.json*. Los factores que anticipamos como principales fuentes de discrepancia en el robot físico son: (i) el ruido y la deriva reales de la odometría del Create 3 frente a la odometría cuasi-perfecta de Gazebo; (ii) la latencia y el *motion blur* de la cámara OAK-D; (iii) los cambios de iluminación del laboratorio sobre la segmentación HSV de los conos; y (iv) el deslizamiento mecánico. Los *gates* de seguridad y la compensación de latencia son las defensas preparadas de antemano contra (i)–(ii).

Limitaciones. (1) El planificador razona sobre el mapa estático, por lo que un obstáculo denso sobre la única ruta produce navegación ineficiente (falta un *costmap local*). (2) El cierre de lazo de la Parte A depende de la re-observación de

marcadores; en tramos largos sin ArUco visible la deriva sólo se corrige al reencontrar un marcador. (3) La validación del sistema completo en hardware está pendiente de la ventana de laboratorio.

VII. CONCLUSIÓN Y TRABAJO FUTURO

Presentamos la memoria técnica de un sistema robótico autónomo completo que integra SLAM visual con Graph SLAM y marcadores ArUco (Parte A), navegación autónoma con localización por filtro de partículas, planificación A^* y una máquina de estados (Parte B), y una capa de misión de exploración informativa con percepción de conos rojos (Parte C). El sistema fue validado en simulación y sobre RosBAGs reales, alcanzando un error de seguimiento del MCL de 0,02–0,18 m y evasión de obstáculos no mapeados sin colisión.

Contribuciones principales:

- Pipeline de Graph SLAM de dos pasadas sobre datos reales, con modelo de medición ArUco caracterizado empíricamente y cierre de lazo por re-observación con triple compuerta.
- Sensor virtual de landmarks con oclusión geométrica, desacoplado de la estimación, que permite portar el Sistema 3 a simulación de forma consistente.
- Arquitectura de misión en capas que separa la decisión de *a dónde* ir (exploración informativa) del *cómo* moverse (FSM de navegación), reutilizando la pila de la Parte B sin modificarla.
- Documentación honesta de las patologías de convergencia y sus correcciones verificadas con datos.

Trabajo futuro: (1) completar la validación sobre el TurtleBot4 físico y el análisis cuantitativo de la brecha sim-to-real; (2) incorporar un *costmap local* para que A^* rodee obstáculos dinámicos en lugar de re-planear contra el mapa estático; (3) añadir *loop closure* geométrico por *scan-matching* para robustecer la Parte A en tramos sin marcadores; (4) explorar una asociación de datos probabilística para los landmarks visuales en escenarios con IDs ambiguos; y (5) endurecer la segmentación de conos ante iluminación variable mediante normalización de color o un detector aprendido.

REFERENCIAS

- [1] S. Thrun, W. Burgard y D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.
- [2] G. Grisetti, R. Kümmerle, C. Stachniss y W. Burgard, “A Tutorial on Graph-Based SLAM,” *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2010.
- [3] S. Garrido-Jurado *et al.*, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [4] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’ Journal of Software Tools*, 2000.
- [5] D. Fox, “Adapting the Sample Size in Particle Filters Through KLD-Sampling,” *Int. Journal of Robotics Research*, vol. 22, no. 12, pp. 985–1003, 2003.
- [6] P. E. Hart, N. J. Nilsson y B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Trans. Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [7] R. C. Coulter, “Implementation of the Pure Pursuit Path Tracking Algorithm,” Carnegie Mellon Univ., Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.

- [8] S. Macenski *et al.*, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.